

Henry Humberto Cruces Castro

Tecnología, costos y diseño

Diapositivas 7–14 · Proyecto «Nova» · InnovVentas · AI-900T00 (SENATI)

Diapositiva 7 — Decisión tecnológica — Azure y alternativas

- El curso es de Azure; la opción ideal era Azure AI Language (CLU).
- Al agotarse el crédito de estudiante, elegimos alternativas de capa gratuita equivalentes: Groq, Render, Neon y Netlify.
- Clave: la arquitectura es desacoplada y migrable a Azure sin reescribir código.

Diapositiva 8 — Stack tecnológico

- Frontend: HTML/CSS/JS vanilla (embebible con un `<script>`).
- Backend: Python + FastAPI. Motor NLP: LLM Groq (Llama 3.3) con las FAQs como contexto.
- Base de datos: PostgreSQL (Neon). Despliegue: Render + Netlify.

Diapositiva 9 — Costos

- Costo del MVP actual: US\$ 0 — todo en capa gratuita.
- Producción estimada en Azure: ≈ US\$ 60/mes (referencial según volumen).
- Desarrollo: equipo de 3 personas, ~4 semanas (costo académico).

Diapositiva 10 — Retorno de inversión (ROI)

- Cómo gana InnovVentas: ventas recuperadas, ahorro en soporte, retención y datos accionables.
- Con costo casi nulo, el payback es inmediato: recuperando 1–2 ventas al mes el bot ya se paga.
- Estimación ilustrativa: recuperar el 5% de carritos abandonados ≈ S/ 29,750/mes adicionales (supuestos declarados).

Diapositiva 11 — Diseño — FAQs por categoría

- La base de conocimiento son 6 categorías: productos, precios, pagos, compra, envíos y soporte.
- Si la consulta no está cubierta, Nova no inventa: deriva a un humano (fallback).

Diapositiva 12 — Diseño — Flujo conversacional

- Flujo: el usuario escribe → Nova procesa (LLM + FAQs) → responde → registra en PostgreSQL → encuesta CSAT.
- Refuerza el manejo del fallback: si no está cubierto, deriva y se registra para mejorar las FAQs.

Diapositiva 13 — Diseño — Arquitectura técnica

- Tres capas: Frontend (Netlify) → Backend FastAPI (Render) → Motor NLP (Groq) + Base de datos (Neon).
- Seguridad: HTTPS, claves solo en el backend, sin datos personales sensibles (Ley N° 29733).

Diapositiva 14 — Diseño — Modelo de datos y entorno

- Entorno con Docker + Docker Compose: un comando levanta el backend y PostgreSQL juntos; mismo entorno en desarrollo y producción (portable, listo para Azure Container Instances).
- init.sql crea el esquema y siembra las 18 FAQs al iniciar la base de datos.
- Modelo de datos: 4 tablas — faqs (conocimiento), sessions, chat_logs (mensaje + intención) y feedback (CSAT) — más vistas SQL para el dashboard.

□ **Transición: «Jean continúa con la implementación, métricas y el demo en vivo.»**

Banco de preguntas y respuestas (blindaje)

Más probables para tu bloque:

❑ **¿Por qué no usaron Azure si el curso es de Azure?**

Era la opción ideal, pero se agotó el crédito de estudiante. Usamos alternativas gratuitas equivalentes y la arquitectura permite volver a Azure sin reescribir código.

❑ **¿Cuál es el retorno de inversión (ROI)?**

El costo de operación es casi nulo, así que el payback es inmediato: recuperando 1–2 ventas al mes ya se paga. Con supuestos conservadores, ~S/ 29,750/mes adicionales.

❑ **¿Cómo gana dinero InnovVentas con el bot?**

Ventas recuperadas, ahorro en soporte (menos tickets), mayor retención y datos accionables del dashboard.

❑ **¿Es seguro? ¿Qué pasa con los datos personales?**

No se almacenan datos sensibles, los logs son anónimos, todo va por HTTPS y las claves viven en el backend. Alineado a la Ley N° 29733 (Perú).

❑ **¿Escala si crece la demanda?**

FastAPI es asíncrono y la arquitectura está desacoplada: migra a Azure Container Instances + Azure DB sin reescribir código.

❑ **¿Y si el LLM inventa información incorrecta?**

Responde solo con las FAQs (grounding) y un prompt estricto; si no está cubierto, deriva a un humano. No improvisa precios ni specs.

❑ **¿Cómo levantaron el entorno de desarrollo?**

Con Docker + Docker Compose: un solo comando levanta el backend y PostgreSQL con el mismo entorno en desarrollo y producción. El esquema y las FAQs se cargan con `init.sql`.

❑ **¿Cuál es el modelo de datos? ¿Qué hace `init.sql`?**

`init.sql` crea 4 tablas en PostgreSQL —`faqs` (conocimiento), `sessions`, `chat_logs` (mensaje + intención + confianza) y `feedback` (CSAT 1–5)— y siembra 18 FAQs, más vistas para el dashboard.

Banco completo (todos deben conocerlo):

¿Cómo se llama el chatbot?

Se llama Nova. Tiene nombre y personalidad (amigable, conciso, español peruano): humaniza la marca y lo diferencia de un chatbot genérico.

¿El bot conoce el stock y el precio real de cada producto?

No en el MVP: Nova atiende FAQs y políticas, y deriva a la ficha del producto. Integrar el inventario en tiempo real es trabajo futuro (decisión de alcance, no una falla).

¿Qué problema concreto resuelve?

El abandono de carrito por falta de respuestas rápidas. Nova responde en el momento de la compra y guía el checkout.

En el demo, ¿por qué tarda la primera respuesta?

Es el plan gratuito de Render que «duerme» tras inactividad (~30 s al despertar). En producción/Azure no ocurre.

¿Cómo se integra al sitio web existente?

Con un widget embebible: dos líneas de `<script>` en cualquier página, y CORS configurado en el backend.

¿Quién mantiene y actualiza las FAQs?

Hoy se editan en la base de datos; el trabajo futuro incluye un panel de administración para hacerlo sin código.

¿Cómo se valida que cumple? (normas y medio ambiente)

Seguridad de la información (HTTPS, datos protegidos) y bajo impacto ambiental por ser una solución cloud serverless, sin hardware adicional.